Apache Flink Kickstarter II **[UNDER CONSTRUCTION]**

> ***On GitHub, Watch → Custom → Releases is the most useful setting if you want to stay updated on when something important is released for this project.***

Apache Flink Kickstarter II is the 2026 evolution of my original Kickstarter project — rebuilt to showcase the cutting edge of **Apache Flink 2.1.x**.

Designed as a hands-on, production-minded accelerator, it brings Flink to life *locally* on **Confluent Platform on Minikube**, while drawing direct comparisons to **Confluent Cloud for Apache Flink** — so you can clearly see what's possible across environments.

Every **example** is delivered end-to-end — from schema design to fully operational streaming pipelines — with implementations in **both Java and Python (when possible)** where it matters, bridging real-world developer workflows with modern streaming architecture.

Table of Contents

- 1.0 Prerequisites
  - 1.1 Confluent Platform on Minikube — Production-Like Streaming, Running Locally
    - 1.1.1 Requirements
  - 1.2 Confluent Cloud Setup
    - 1.2.1 Requirements
- 2.0 The Examples
  - 2.1 Apache Flink User-Defined Functions (UDF)
    - 2.1.1 Process Table Functions (PTF)
      - 2.1.1.1 Limitation(s)
- 3.0 Debugging the Examples
  - 3.1 Apache Flink UDF Debugging with Java Debug Wire Protocol (JDWP)
    - 3.1.1 Process Table Functions (PTF)
      - 3.1.1.1 Debugging the `row-driven` PTF using set semantics (`UserEventEnricher`)
      - 3.1.1.2 Debugging the `row-driven` PTF using row semantics (`OrderLineExpander`)
      - 3.1.1.3 Debugging the `timer-driven` PTFs (`SessionTimeoutDetector`, `AbandonedCartDetector`, `PerEventFollowUp`, and `SlaMonitor`)
- 4.0 Resources
  - 4.1 Confluent for Kubernetes (CfK)
  - 4.2 Confluent Platform for Apache Flink
  - 4.3 Confluent Cloud for Apache Flink (CCAF)

1.0 Prerequisites

1.1 Confluent Platform on Minikube — Production-Like Streaming, Running Locally

To ***run***, ***test***, and ***debug*** Apache Flink like a production engineer, this project provides a full Confluent Platform stack running locally on Minikube — no cloud required.

You get a production-like environment on your laptop:

- **Confluent Platform** (KRaft mode) via Confluent for Kubernetes (CFK)
- **Apache Flink 2.1.1** via the Confluent Flink Kubernetes Operator 1.130
- **Confluent Manager for Apache Flink (CMF) 2.1** for Flink environment management

> Build locally. Debug with confidence. Deploy to production-ready environments.

To get started with working with Confluent Platform on Minikube using the `Makefile` automation, click here.

1.1.1 Requirements

To run this project, you'll need **macOS (with Homebrew)** or **Linux (with apt-get)**.

The full stack — **Minikube + Confluent Platform + Flink + CMF** — is resource-intensive and designed to mirror a production-like environment. The following defaults are recommended:

Resource	Default
CPUs	6
Memory	20 GB
Disk	50 GB

> These settings ensure stable performance across all components. You can tune them if needed, but lower resources may lead to pod restarts or degraded performance.

1.2 Confluent Cloud Setup

In addition to running the examples locally, you also have the ability to run the examples in Confluent Cloud. The examples are designed to be as close as possible to the local Confluent Platform setup, so you can easily compare and contrast the two environments.

1.2.1 Requirements

Before you begin, ensure you have access to the following cloud accounts:

- **Confluent Cloud Account** — for Kafka, Schema Registry, and Flink resources
- **Terraform Cloud Account** — for automated infrastructure provisioning

Make sure the following tools are installed on your local machine:

- **Java JDK 21** — for building Flink UDFs
- **Gradle 9.4.1 or higher** — for building Flink UDFs
- **Terraform CLI version 1.13.0 or higher** — for deploying infrastructure to Confluent Cloud

2.0 The Examples

Once you've set up **Confluent Platform on Minikube** or created your **Confluent Cloud** account, you're ready to try the examples.

2.1 Apache Flink User-Defined Functions (UDF)

2.1.1 Process Table Functions (PTF)

▶ ***What are PTFs?***

PTFs are a special type of Apache Flink UDF that offers stateful, timer-aware processing capabilities directly within Flink SQL. PTFs can be either `row-driven` (invoked for each input row) or `timer-driven` (triggered based on timers you set in your code).

▶ ***Why use PTFs?***

PTF UDFs are ideal when you need memory across rows, respond to time passing — not just on arriving data, or you want to implement complex event processing (CEP) patterns that are difficult to express in pure Flink SQL.

▶ ***When do you use PTFs?***

PTFs are used when your use case requires state and/or timers that go beyond what standard Flink SQL can handle. For example: **Stateful Enrichment with or without External Lookups**, **Per-Row Stateful Transformation**, **Complex Conditional Routing and/or Filtering**, **Session timeout**, **Abandoned cart**, **Device heartbeat monitoring**, **Per-event follow-up**, **SLA monitoring**, **Delayed side-effects**, and more.

▶ ***Where do you use PTFs?***

You write PTF UDFs as Java classes, deploy them as JAR files, and run them within your Flink SQL queries.

▼ ***How are examples of PTFs put into practice?***

Type	Purpose	Confluent Platform on Minikube	Confluent Cloud
PTF UDF-type (`row-driven`)	Walks through both **local** and cloud environments *building*, *deploying*, and *testing* two `row-driven` **PTF UDFs** (no timers) bundled in one JAR that illustrate both `ArgumentTrait` modes: **User Event Enricher** (`SET_SEMANTIC_TABLE` — enriches Kafka user events with per-user session tracking using keyed state) and **Order Line Expander** (`ROW_SEMANTIC_TABLE` — stateless one-to-many expansion of an order row into individual line items).	`CP Deploy`	`CC Deploy`

Type	Purpose	Confluent Platform on Minikube	Confluent Cloud
PTF UDF-type (`timer-driven`)	Walks through both **local** and cloud environments *building*, *deploying*, and *testing* four `timer-driven` **PTF UDFs** bundled in one JAR: **Session Timeout Detector** (named timers using the inactivity pattern), **Abandoned Cart Detector** (named timers using the inactivity pattern for e-commerce), **Per-Event Follow-Up** (unnamed timers using the scheduling pattern), and **SLA Monitor** (unnamed timers using the scheduling pattern).	`CP Deploy`	`CC Deploy`

> ⏳ **PTF Rules of Thumb: The Hourglass Pattern**

2.1.1.1 Limitation(s)

- Why `@StateHint` POJO with `Map` or `List` Are Sensitive to "Extremely Large State"

3.0 Debugging the Examples

3.1 Apache Flink UDF Debugging with Java Debug Wire Protocol (JDWP)

You can attach your IDE's debugger (VS Code or IntelliJ IDEA) to a running Flink TaskManager and *hit breakpoints inside your UDF code* — even though it's executing on a remote Java Virtual Machine (JVM) inside Kubernetes. The `FlinkDeployment` Custom Resource (CR) already has **Java Debug Wire Protocol (JDWP)** enabled, and debug configurations are pre-wired for both VS Code and IntelliJ IDEA.

Prerequisites: The Confluent Platform and Flink stack must be running (`make cp-up && make flink-up`), and your UDF must be deployed.

3.1.1 Process Table Functions (PTF)

3.1.1.1 Debugging the `row-driven` PTF using set semantics (`UserEventEnricher`)

> For the full deep-dive, see Remote Debugging `row-driven` Flink PTF UDFs. The same guide also covers debugging `OrderLineExpander` (§1.3).

Deploy first: `make deploy-cp-ptf-udf-row-driven`, and then:

▶ 1. Set a breakpoint

Open `UserEventEnricher.java` and click in the gutter at the first line of the `eval()` method:

```
String eventType = input.getFieldAs("event_type");
```

▶ 2. Attach the debugger

Select the **"Attach to Flink TaskManager (Row-Driven)"** configuration and start debugging. The IDE will automatically port-forward to the TaskManager pod and attach to the JDWP agent on port `5005`.

- **VS Code:** Open the **Run and Debug** panel (⇧⌘D), select the configuration from the dropdown, and press **F5**
- **IntelliJ IDEA:** Open the **Run/Debug Configurations** dropdown (top-right toolbar), select the configuration, and click **Debug** (^D / Shift+F9)

▶ 3. Send a test event

Produce a single JSON message to the `user_events` topic to trigger the breakpoint:

```
make produce-user-events-record
```

▶ 4. Debug

Your IDE will pause at your breakpoint. You can inspect `input`, `state`, and local variables, step through the session logic, and watch `state.sessionId` and `state.eventCount` update as you step over lines.

3.1.1.2 Debugging the `row-driven` PTF using row semantics (`OrderLineExpander`)

> For the full deep-dive, see Remote Debugging `row-driven` Flink PTF UDFs (§1.3).
>
> The `OrderLineExpander` ships in the **same uber JAR** as `UserEventEnricher`, so the same `make deploy-cp-ptf-udf-row-driven` command and the same **"Attach to Flink TaskManager (`row-driven`)"** debug configuration are used. The deploy script registers both PTFs as separate Flink SQL functions and starts an `INSERT INTO orders_expanded SELECT ... FROM TABLE(order_line_expander(input => TABLE orders))` pipeline alongside the user-event enrichment job.

Deploy first: `make deploy-cp-ptf-udf-row-driven`, and then:

▶ 1. Set a breakpoint

Open `OrderLineExpander.java` and click in the gutter at the first line of the `eval()` method:

```
String orderId  = input.getFieldAs("order_id");
```

Or, to inspect the per-item emission, set a breakpoint inside the expansion `for` loop on the `collect(Row.of(...))` call.

▶ 2. Attach the debugger

Use the **same "Attach to Flink TaskManager (Row-Driven)"** configuration as `UserEventEnricher` — both PTFs run in the same TaskManager pod from the same JAR. The IDE will automatically port-forward to the TaskManager pod and attach to the JDWP agent on port `5005`.

- **VS Code:** Open the **Run and Debug** panel (⇧⌘D), select the configuration from the dropdown, and press **F5**
- **IntelliJ IDEA:** Open the **Run/Debug Configurations** dropdown (top-right toolbar), select the configuration, and click **Debug** (^D / Shift+F9)

▶ 3. Send a test order

Produce a single JSON order to the `orders` topic to trigger the breakpoint:

```
make produce-orders-record
```

The sample record has three items in its comma-separated list, so `eval()` will fire once and the `for` loop inside it will emit three rows.

▶ 4. Debug

Your IDE will pause at your breakpoint. Inspect `input`, the parsed `itemParts` and `quantityParts` arrays, and step through the `for` loop watching `i`, `itemName`, `qty`, and the `collect()` call. Notice that:

- There is **no `state` parameter** — row semantics forbids `@StateHint`, so nothing is preserved between rows.
- There is **no `Context` parameter** — no timers or keyed-state services are accessible.
- A single `eval()` call emits **multiple output rows** via repeated `collect()` calls — this is the canonical one-to-many pattern that distinguishes row-semantic PTFs from scalar UDFs.

> **Row-semantic debugging tip:** Because `OrderLineExpander` is stateless, every input row hits `eval()` independently and the framework is free to distribute rows across virtual processors. If you produce multiple orders in quick succession your breakpoint may fire on any TaskManager slot — and rows from different orders may interleave in arbitrary order. Use the `order_id` field in the watch panel to keep track of which row you're inspecting.

3.1.1.3 Debugging the `timer-driven` PTFs (`SessionTimeoutDetector`, `AbandonedCartDetector`, `PerEventFollowUp`, and `SlaMonitor`)

> For the full deep-dive, see Remote Debugging `timer-driven` Flink PTF UDFs.

Deploy first: `make deploy-cp-ptf-udf-timer-driven`, and then:

▶ 1. Set a breakpoint

Open `SessionTimeoutDetector.java` and click in the gutter at the first line of the `eval()` method:

```
String eventType = input.getFieldAs("event_type");
```

Or, to debug the unnamed timer UDF, open `PerEventFollowUp.java` and set a breakpoint at:

```
String eventType = input.getFieldAs("event_type");
```

Or, to debug the Abandoned Cart Detector, open `AbandonedCartDetector.java` and set a breakpoint at:

```
String action = input.getFieldAs("action");
```

Or, to debug the SLA Monitor, open `SlaMonitor.java` and set a breakpoint at:

```
String status = input.getFieldAs("status");
```

Or, to debug a timer callback, set a breakpoint in `onTimer()` of any UDF.

▶ 2. Attach the debugger

Select the **"Attach to Flink TaskManager (Timer-Driven)"** configuration.

- **VS Code:** Open the **Run and Debug** panel (⇧⌘D), select the configuration from the dropdown, and press **F5**
- **IntelliJ IDEA:** Open the **Run/Debug Configurations** dropdown (top-right toolbar), select the configuration, and click **Debug** (^D / Shift+F9)

▶ 3. Send a test event

Produce a single JSON message to the `user_activity` topic to trigger the breakpoint:

```
make produce-user-activity-record
```

▶ 4. Debug

Your IDE will pause at your breakpoint. Inspect `input`, `state`, and local variables, step through the timer registration logic, and watch `state.eventCount` and `state.lastEventType` update as you step over lines.

> **Timer debugging tip:** Timers fire when the watermark advances past the timer's registered time. While paused at a breakpoint, watermarks don't advance, so `onTimer()` won't fire until you resume execution and let the watermark progress. For the unnamed timer UDFs (`PerEventFollowUp` and `SlaMonitor`), note that `onTimer()` fires once per event — not once per partition key. Both the `AbandonedCartDetector` and `SlaMonitor` demonstrate conditional output: `onTimer()` only emits if the cart wasn't checked out or the request wasn't resolved, respectively.

4.0 Resources

4.1 Confluent for Kubernetes (CfK)

- Manage Confluent Platform with Confluent for Kubernetes
- Minikube

4.2 Confluent Platform for Apache Flink

- Stream Processing with Confluent Platform for Apache Flink
- Architecture and Features of Confluent Platform for Apache Flink
- Get Started with Confluent Platform for Apache Flink

4.3 Confluent Cloud for Apache Flink (CCAF)

- Stream Processing with Confluent Cloud for Apache Flink
- Get Started with Confluent Cloud for Apache Flink